

ESTRUTURA DE DADOS

Alien Dictionary

Descobrimos a ordem de um alfabeto desconhecido com grafos e ordenação topológica

Trabalho em grupo • Bruno Mascioli, Felipe Santos, Heitor Lemes, Lucas Santos, Tiago Catoia



1 Introdução e contextualização do problema

O que é o problema?

- Recebemos uma lista de palavras que já está ordenada segundo o alfabeto de uma língua alienígena desconhecida.
- O objetivo é descobrir qual é a ordem das letras desse alfabeto, sem conhecer nenhuma regra.
- É um problema clássico de estrutura de dados: por trás de uma pergunta sobre strings, existe um problema de grafos.
- A mesma ideia poderia ordenar qualquer conjunto de símbolos, não precisam ser letras do nosso alfabeto.

EXEMPLO

Entrada (ordenada):

```
["wrt", "wrf", "er", "ett", "rftt"]
```

Saída (ordem das letras):

```
"wertf"
```

Comparando palavras vizinhas, letra por letra, conseguimos inferir quem vem antes de quem.

1 Onde esse problema aparece?

Contexto em entrevistas técnicas

NÍVEL DE DIFICULDADE

Easy

Medium

Hard

- LeetCode #269 — classificado como Hard.
- Hoje faz parte do banco de questões Premium (assinatura paga).
- A dificuldade não está na sintaxe, e sim em modelar o problema como um grafo antes de programar.

EMPRESAS QUE JÁ COBRARAM ESSA QUESTÃO

Google

Meta (Facebook)

Amazon

Microsoft

Airbnb

Bloomberg

Uber

Com base em relatos de candidatos no LeetCode Discuss, Glassdoor e fóruns como Blind.

PARA QUAIS VAGAS?

Entrevistas técnicas de Software/Backend Engineer, tipicamente em vagas de nível pleno a sênior — avalia raciocínio sobre grafos, não a decoreba de um algoritmo.

2 Conceitos teóricos necessários

Grafos e Ordenação Topológica

1

Grafo dirigido (DAG)

Um conjunto de nós conectados por arestas com direção, sem ciclos. Cada letra do alfabeto vira um nó do grafo.

2

Arestas de precedência

Comparamos palavras adjacentes: a primeira letra diferente entre elas indica que uma vem antes da outra no alfabeto.

3

Ordenação topológica

Organiza os nós do grafo em uma sequência linear que respeita todas as relações de precedência essa sequência é a resposta.

Deteção de erro: se o grafo tiver um ciclo, não existe ordenação válida, é assim que identificamos entradas inconsistentes.

3 Construção do algoritmo passo a passo

1

Montar o grafo

Cada letra única vira um nó. Para cada par de palavras adjacentes, encontrar a primeira letra diferente e criar uma aresta de precedência.

2

Calcular grau de entrada

Contar, para cada letra, quantas arestas chegam até ela (in-degree).

3

Tratar casos inválidos

Se uma palavra maior aparece como prefixo antes da menor (ex: "abc" antes de "ab"), ou se há um ciclo no grafo, a ordem é impossível.

4

Gerar a ordenação

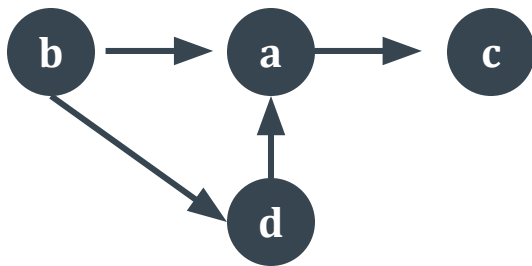
Aplicar ordenação topológica (fila de graus zero) para obter a sequência final das letras do alfabeto alienígena.

3 Exemplo aplicado

Usando ["baa", "abcd", "abca", "cab", "cad"]

Par comparado	1ª letra diferente	Aresta gerada
baa , abcd	b vs a	b → a
abcd , abca	d vs a	d → a
abca, cab	a vs c	a → c
cab , cad	b vs d	b → d

Grafo resultante (encadeamento das arestas):



In-degree (evolução no Algoritmo de Kahn):

Inicial: {"b":0, "d":1, "a":2, "c":1}

1. Pop 'b' → d:0, a:1 (add 'd')

2. Pop 'd' → a:0 (add 'a')

3. Pop 'a' → c:0 (add 'c')

4. Pop 'c' → fila vazia

Final: {"b":0, "d":0, "a":0, "c":0}

ORDEM FINAL

"bdac"

4 Implementação em código

Algoritmo de Kahn (BFS) — Python

```
1 def alien_order(words):
2     grafo = {c: set() for w in words for c in w}
3     grau_entrada = {c: 0 for c in grafo}
4     for w1, w2 in zip(words, words[1:]):
5         for c1, c2 in zip(w1, w2):
6             if c1 != c2:
7                 if c2 not in grafo[c1]:
8                     grafo[c1].add(c2)
9                     grau_entrada[c2] += 1
10                break
11         else:
12             if len(w2) < len(w1):
13                 return "" # prefixo inválido
14     fila = [c for c in grafo if grau_entrada[c] == 0]
15     ordem = []
16     while fila:
17         c = fila.pop()
18         ordem.append(c)
19         for viz in grafo[c]:
20             grau_entrada[viz] -= 1
21             if grau_entrada[viz] == 0:
22                 fila.append(viz)
23     if len(ordem) < len(grafo):
24         return "" # ciclo: ordem impossível
25     return "".join(ordem)
```

Como funciona

- Monta o grafo comparando cada par de palavras vizinhas.
- Inicia a fila com as letras de grau de entrada zero.
- Remove letras da fila, reduzindo o grau de seus vizinhos.
- Se sobrar alguma letra fora da ordem final, há um ciclo.

Veja a execução completa desse código, linha a linha, no próximo slide.

4

Passo a passo da execução

Dry-run do BFS com o exemplo ["wrt", "wrf", "er", "ett", "rftt"]

Passo	Fila	Letra removida	Ordem parcial
0	[w]	—	""
1	[e]	w	"w"
2	[r]	e	"we"
3	[t]	r	"wer"
4	[f]	t	"wert"
5	[]	f	"wertf"

Por que a fila nunca tem mais de um elemento?

Nesse exemplo específico, cada letra aponta para exatamente uma próxima letra, o grafo forma uma cadeia simples. Em casos reais, várias letras podem entrar na fila ao mesmo tempo.

Leitura do código

- `fila.pop()` remove e retorna a letra da vez.
- `ordem.append(c)` registra essa letra na resposta final.
- `grau_entrada[viz] -= 1` libera o próximo nó quando seu grau chega a zero.
- Ao final, `ordem` contém a sequência completa: "wertf".

5

Complexidade, aplicações e conclusão

$O(C)$

Tempo — C é o total de caracteres em todas as palavras (percorremos a lista uma única vez para mapear as letras)

$O(U + \min(U^2, N))$

Espaço — U é o nº de letras únicas (nós do grafo) e N a quantidade de palavras.

O segundo termo é o limite de arestas, que não passa das combinações possíveis de letras (U^2), nem do total de palavras comparadas (N).

Caso de erro: entradas com ciclo ou prefixo fora de ordem retornam string vazia.

Aplicações reais da ordenação topológica

- Gerenciadores de pacotes: resolver dependências entre bibliotecas (npm, pip).
- Compiladores: definir a ordem correta de compilação de módulos.
- Currículos acadêmicos: ordenar disciplinas que possuem pré-requisitos.
- Build systems (Make, etc.): decidir em que ordem compilar arquivos alterados.
- O Algoritmo de Kahn se mostra uma solução intuitiva, que permite resolver o problema enquanto lida naturalmente com validações como detecção de ciclos.

Obrigado!

Perguntas sobre o Alien Dictionary e ordenação topológica?